MODULE 3 · 10 HOURS

Retrieval-Augmented Generation (RAG)

Unit 1 — Introduction to RAG & Document Preprocessing

Motivation | Hallucination Reduction | Domain Adaptation | Naive RAG | Advanced RAG | Modular RAG

Document Loading | Chunking

ROADMAP

Three Building Blocks of Unit 1

From why RAG exists, to the architectures that implement it, to how documents get ready to be searched.

1

Introduction to RAG

Why it exists: hallucination reduction and domain adaptation

2

RAG Architectures

Naive, Advanced, and Modular RAG pipelines compared

3

Document Loading & Preprocessing

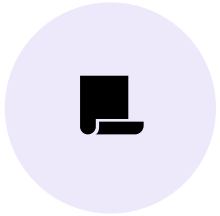
Getting raw documents ready to be searched

INTRODUCTION · PART 1

What Is RAG?

RAG pairs an AI model with a search step, so it answers using real retrieved documents instead of only what it memorized during training.

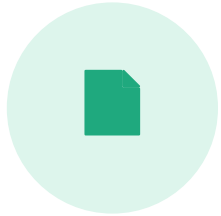
✗ **LLM ALONE**



"What's our current return policy?"

Answers only from memory — may be outdated, generic, or simply invented.

✓ **LLM + RETRIEVAL (RAG)**



"What's our current return policy?"

Searches real company documents first, then answers using what it actually found.

Why RAG? The Motivation

Large language models have three built-in limitations that retrieval directly solves.

Knowledge Cutoff

The model only knows what existed up to its training date — nothing after that, unless it can search.

Private & Custom Data

A model was never trained on your company's internal documents, tickets, or wikis.

Costly to Retrain

Retraining a model on new information is slow and expensive. Updating a document store is instant.

RAG addresses all three at once: point the model at current, private, or specialized documents — no retraining required.

Hallucination Reduction

A hallucination is when an AI confidently states something false. RAG reduces this by grounding answers in real retrieved text.

Question: *"How many vacation days do new employees get in their first year?"*

WITHOUT RAG

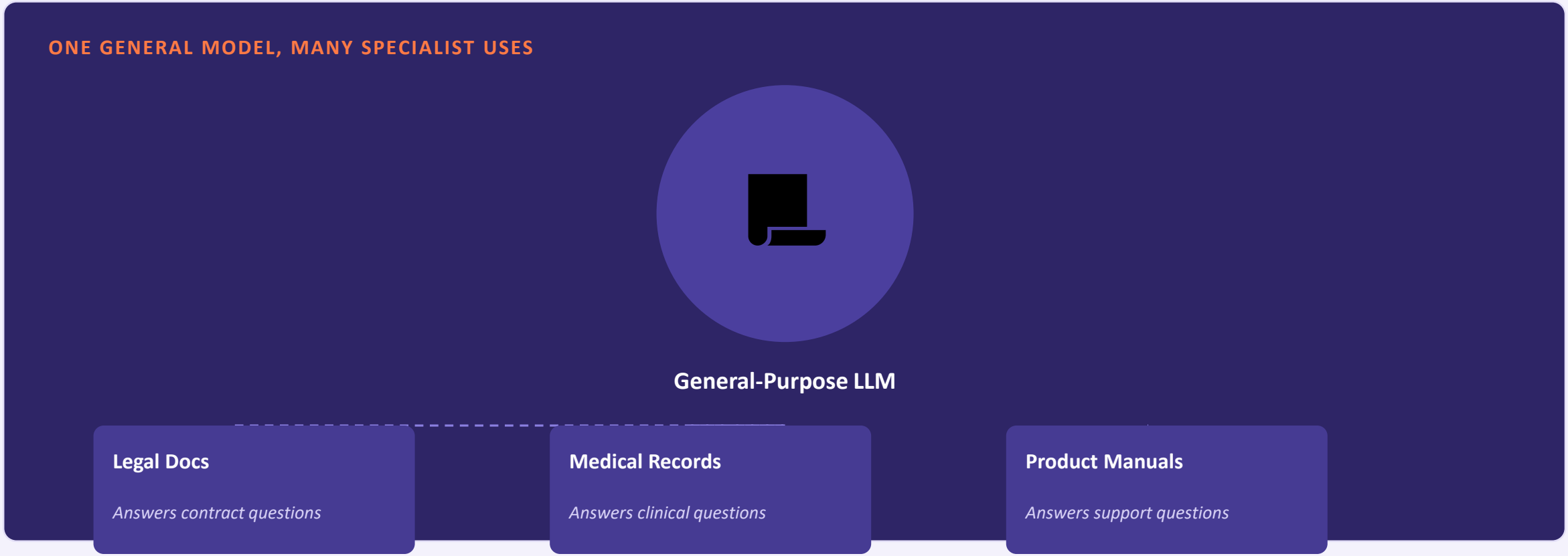
"New employees typically get 15 days of paid vacation." (Sounds confident — but this is a guess. The actual policy is 12 days plus 3 floating holidays.)

WITH RAG

Retrieves the actual HR policy document, then answers: "12 days of paid vacation, plus 3 floating holidays, per the 2026 HR handbook." (Grounded in a real source.)

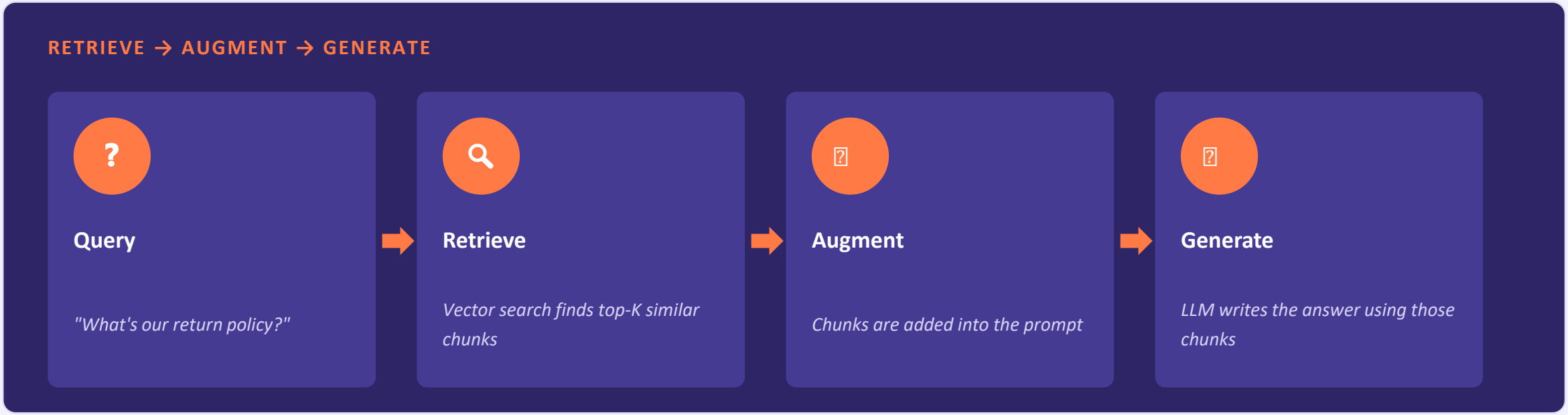
Domain Adaptation

RAG lets a general-purpose model act like a specialist in a narrow domain — just by pointing it at the right documents, with no retraining.



Naive RAG

The simplest RAG pipeline: retrieve the most similar chunks, add them to the prompt, and generate an answer.



Great for: quick prototypes and simple Q&A over a small, clean document set. Fast to build, easy to understand.

Where Naive RAG Falls Short

The simple retrieve-then-generate pipeline breaks down as documents and questions get more complex.

Irrelevant Chunks

Similarity search can pull chunks that look related but don't actually answer the question.

No Re-ranking

The first chunks found aren't always the most useful ones — naive RAG doesn't double-check.

Context Overload

Too many chunks crowd the prompt; too few leave out key information.

Single-Shot Retrieval

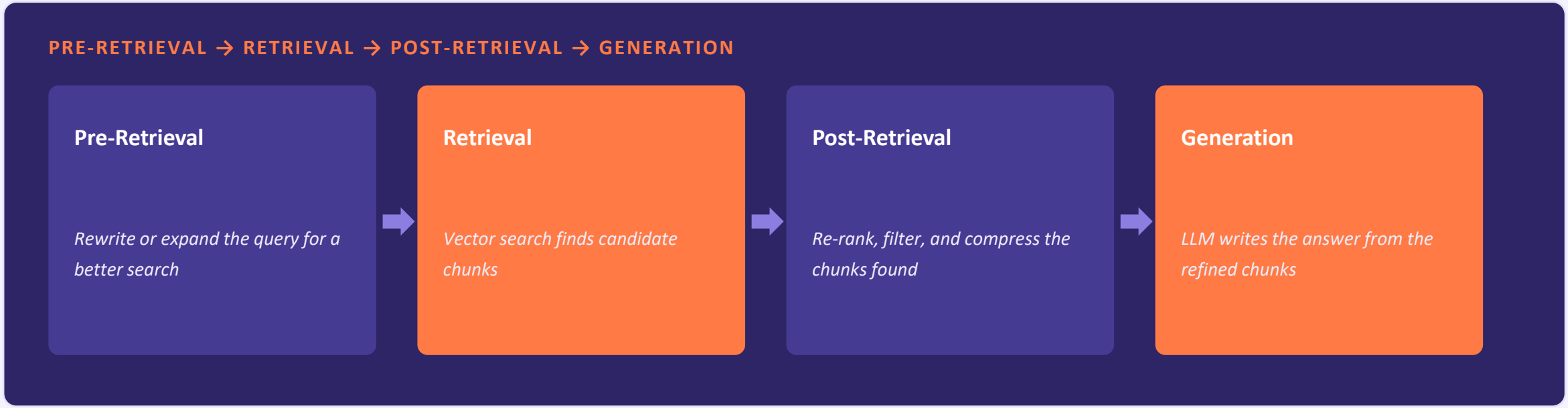
If the first search misses, there's no attempt to search again or rephrase.

These gaps are exactly what Advanced and Modular RAG are designed to fix.

ARCHITECTURES · PART 2

Advanced RAG

Adds optimization steps before and after retrieval to fix naive RAG's weaknesses.



Great for: production systems where answer accuracy matters and documents are large, messy, or numerous.

Advanced RAG Techniques

Query Rewriting

Rephrases a vague or poorly worded question into one better suited for search.

Re-ranking

Re-scores the retrieved chunks with a more accurate (but slower) model, and reorders them.

Compression

Trims each chunk down to just the sentences that are actually relevant.

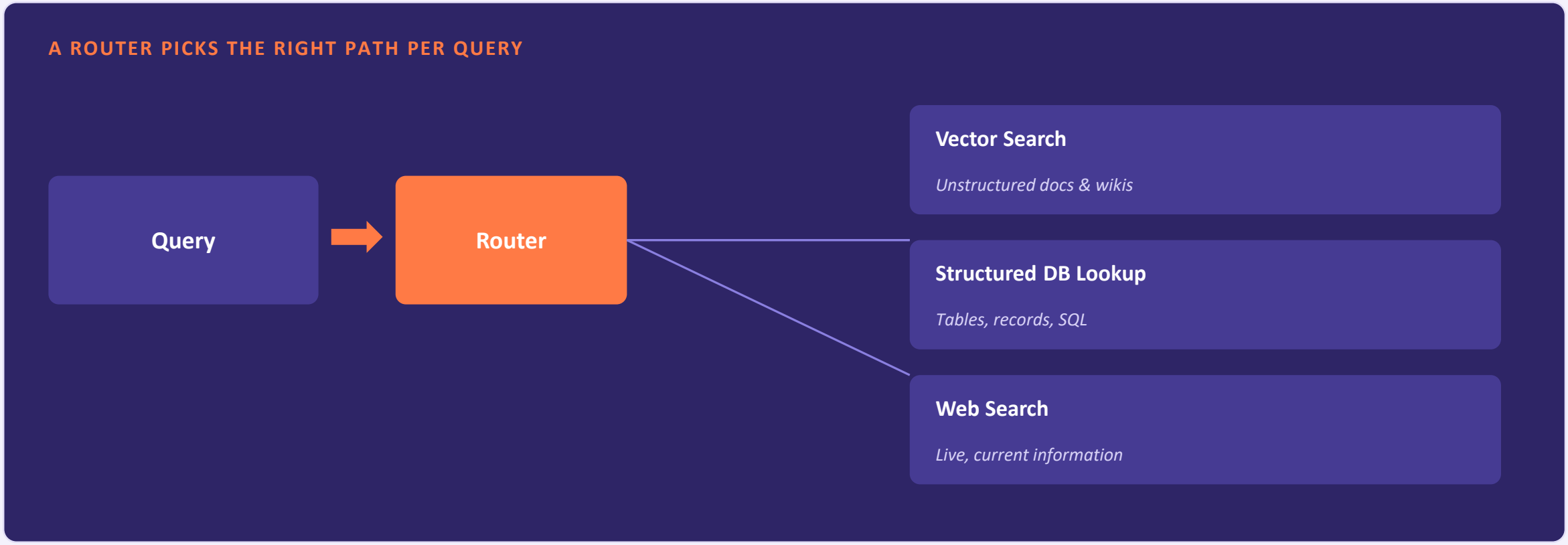
Filtering

Drops chunks that fall below a confidence or relevance threshold entirely.

Example: a user asks "policy on returns?" — query rewriting expands this to "What is the company's product return and refund policy?" before searching.

Modular RAG

Breaks RAG into swappable, reusable modules — a router decides which retrieval path fits each query, instead of one fixed pipeline.



This flexibility makes modular RAG well suited to complex systems pulling from many different kinds of sources.

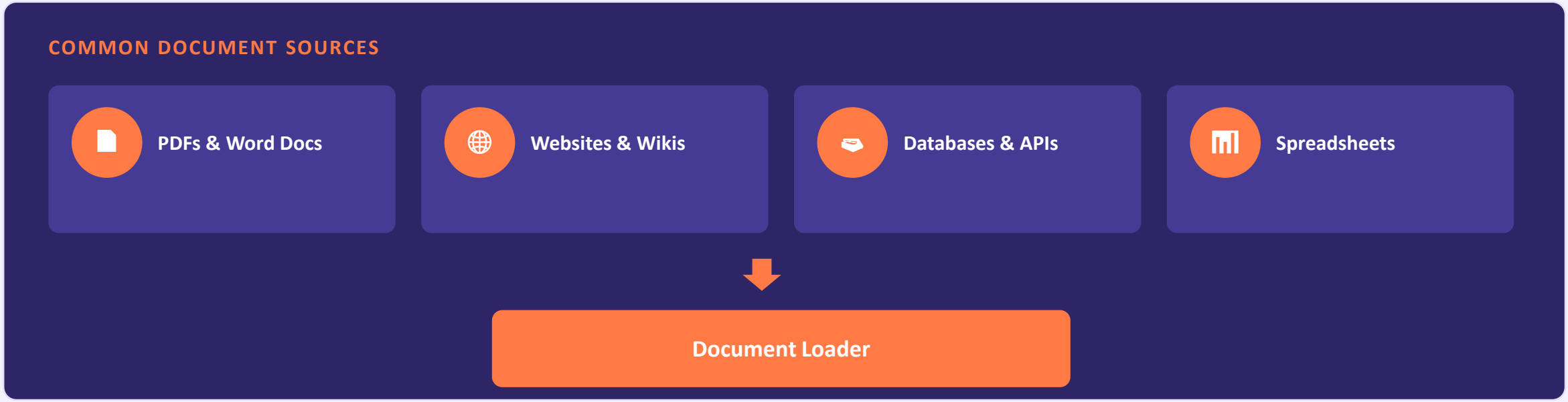
Naive vs. Advanced vs. Modular RAG

Architecture	Pipeline Shape	Best For	Complexity
Naive RAG	Fixed retrieve-then-generate	Quick prototypes, simple Q&A	Low
Advanced RAG	Optimized with pre/post-retrieval steps	Production systems needing accuracy	Medium
Modular RAG	Flexible, swappable components with routing	Complex, multi-source systems	High

Start naive to prove the concept, add advanced techniques as accuracy needs grow, and go modular when you have multiple data sources to orchestrate.

Document Loading

Before retrieval can work, documents must be pulled in from wherever they live and converted into plain text.



Output: plain text, extracted from each source, ready for cleaning and chunking — regardless of the original file format.

Chunking

Documents are too long to search as a whole, so they're split into smaller pieces called chunks.

ONE LONG DOCUMENT

Employee Handbook.pdf — 42 pages of policies, benefits, and procedures

SPLIT INTO CHUNKS

Chunk 1

“Vacation policy: 12 days plus 3 floating holidays...”

Chunk 2

“Return policy: items may be returned within 30 days...”

Chunk 3

“Remote work guidelines: employees may work from home...”

Chunking Strategies

How you cut a document into chunks has a big impact on what retrieval can find later.

Fixed-Size Chunking

Splits text every N words or characters. Simple and fast, but can cut sentences in half.

Overlapping Chunks

Neighboring chunks share a small overlap, so context isn't lost right at the boundary.

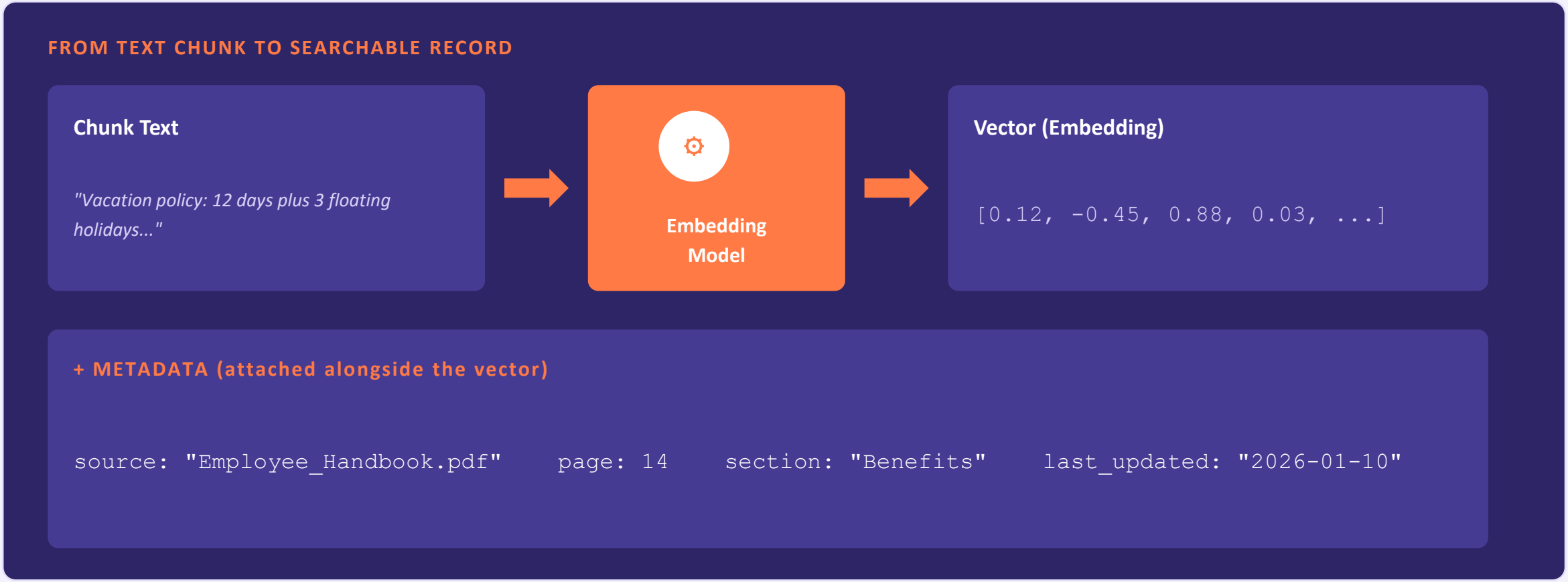
Semantic Chunking

Splits at natural boundaries — paragraphs or sections — instead of arbitrary cut points.

Most real systems use overlapping, semantically-aware chunks — a small amount of overlap plus natural boundaries balances accuracy and simplicity.

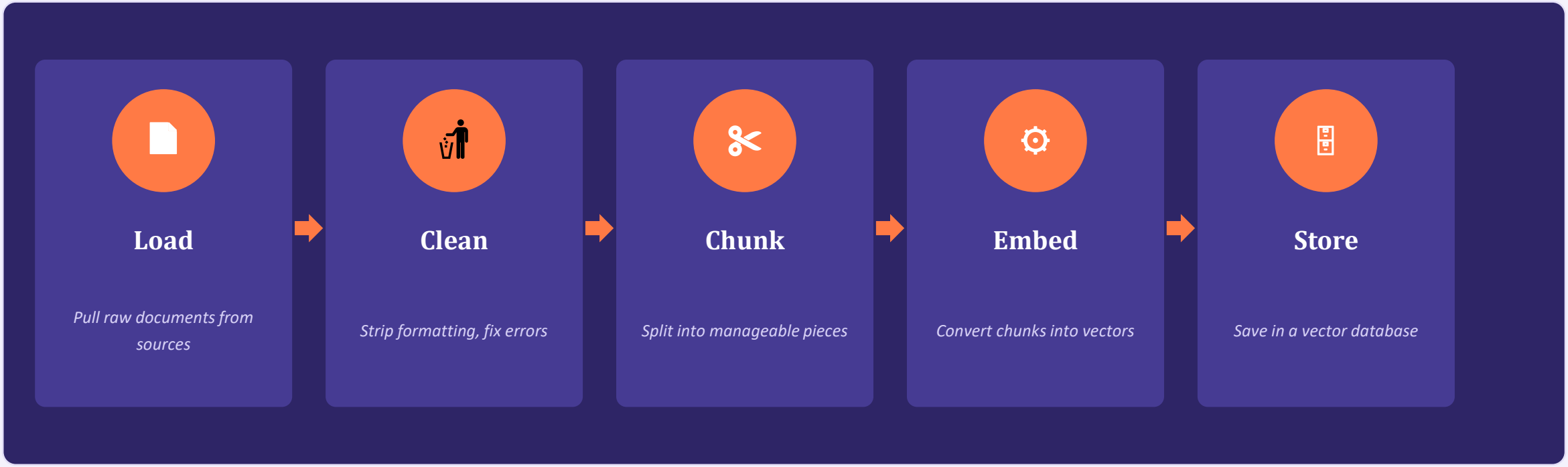
Embeddings & Metadata

Each chunk is converted into an embedding — a list of numbers capturing its meaning — and tagged with metadata for filtering and citation.



The Full Preprocessing Pipeline

This is the foundation that RAG's retrieval step searches over — built once, before any question is ever asked.



Once stored, this vector database is what every future retrieval step searches against.

RECAP

Putting It All Together

1

What Is RAG?

Retrieval + generation, grounded in real docs

2

Motivation

Fixes knowledge cutoffs & private data gaps

3

Hallucination Reduction

Answers grounded in retrieved text

4

Domain Adaptation

One model, many specialist uses

5

Naive RAG

Simple retrieve-then-generate pipeline

6

Advanced RAG

Pre/post-retrieval optimization steps

7

Modular RAG

Swappable components, routed per query

8

Document Preprocessing

Load → Clean → Chunk → Embed → Store

RAG turns a general AI model into one that answers with real, current, and specialized knowledge — without retraining.



Questions & Discussion

Next: Unit 2 will cover retrieval methods and vector databases in depth.